

Research Proposal: Weight Space Alignment for Continual Learning in Robotics

Abstract

Recent work on weight-space representation learning (WeightCLIP) has shown that contrastively aligning a population of trained neural network weights against the datasets that produced them yields a latent space geometrically organized by task semantics. Furthermore, recent work on VLA weight generation (WIZARD) has shown that LoRA adapters for a frozen Vision-Language-Action (VLA) model can be synthesized directly from a task prompt via a single forward pass through a tensorized, modality-factorized weight representation. However, this has been done without any contrastive structuring of that representation, and without evaluation under a sequential, growing task stream.

Our proposal suggests:

1. Aligning each of the three VLA component tokens against a modality-specific task prompt using an independently-weighted contrastive objective.
2. Exploiting the resulting geometric structure of the aligned weight-space representation in two ways during sequential continual learning:
 - Predicting behavioral forgetting from latent drift alone (avoiding expensive simulations).
 - Acting as a differential regularizer that constrains fragile (vision/language) components more tightly than robust (action) components during latent-space refinement.

1. Methodology

1.1 Component-Factorized Tokenization

Given a task k , its LoRA update to the frozen VLA is decomposed by architectural role rather than treated as one undifferentiated parameter block:

$$\Delta W^{(k)} \in \mathbb{R}^{L \times 3 \times r \times H}$$

where L indexes layers, r is the LoRA rank, H is the hidden dimension, and the size-3 axis separates the vision, language, and action sub-modules. Each modality slice is flattened into a token stream:

$$Z_{\text{vis}}^{(k)}, Z_{\text{lang}}^{(k)}, Z_{\text{act}}^{(k)}$$

Each stream is passed through a shared weight autoencoder (g_ϕ, h_ψ) —a transformer encoder/decoder pair following the SANE/WeightCLIP tokenization convention—producing compressed latent token sequences $Z_m^{(k)} = g_\phi(\Delta W_m^{(k)})$ for $m \in \{\text{vis}, \text{lang}, \text{act}\}$.

This factorization reuses WIZARD’s modality-tensorized representation of LoRA updates, but introduces a novel training paradigm beyond a direct reconstruction objective.

1.2 Modality-Specific Contrastive Alignment

For each modality m , we define a dedicated prompt encoder producing an embedding e_m from task-specific evidence:

- e_{vis} : A DeepSets encoder over a small set of demonstration frames.
- e_{lang} : A sentence encoder over the task’s language instruction.
- e_{act} : A set-encoder over summary statistics of the task’s action-chunk distribution.

Replacing WeightCLIP’s single dataset-alignment objective, we define three independent bidirectional contrastive losses (one per modality), each with its own learnable temperature τ_m :

$$\mathcal{L}_{\text{align}}^{(m)} = -\frac{1}{BL} \sum_{i=1}^B \sum_{t=1}^L \log \frac{\exp\left(\tilde{s}\left(z_{m,t}^{(i)}, e_m^{(i)}\right)\right)}{\sum_{j=1}^B \exp\left(\tilde{s}\left(z_{m,t}^{(i)}, e_m^{(j)}\right)\right)} + (\text{reverse direction})$$

where $\tilde{s}(a, b) = \tau_m^{-1} \frac{a^\top b}{\|a\| \|b\|}$, and B is the batch size over the model-zoo population (see Section 4.2). The total training objective for the weight autoencoder combines reconstruction with all three alignment terms:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{recon}} + \lambda_{\text{vis}} \mathcal{L}_{\text{align}}^{(\text{vis})} + \lambda_{\text{lang}} \mathcal{L}_{\text{align}}^{(\text{lang})} + \lambda_{\text{act}} \mathcal{L}_{\text{align}}^{(\text{act})}$$

Prompt encoders are trained jointly (unfrozen) with an auxiliary task-classification head on each, mirroring WeightCLIP’s dataset-encoder supervision.

1.3 Differential Regularization During Sequential Adaptation

When adapting to task k in a continual stream, an initial latent $Z_m^{(0)}$ is obtained via retrieval-augmented mapping (nearest-neighbor task prompt $\rightarrow$ linear or memory-bank mapper, per WeightCLIP §3.4). It is then refined against a small amount of task- k data through the decoder:

$$\mathcal{L}_{\text{ref}}(Z) = \mathcal{L}_{\text{task}}\left(h_\psi(Z)\right) + \sum_m \gamma_m \|\Pi_{\text{shell}}(Z_m) - Z_m^{(0)}\|_F^2$$

where Π_{shell} is the hyperspherical-shell projection (WeightCLIP §3.5), and γ_m is a per-component regularization coefficient. Motivated directly by the vision-language-dominant forgetting asymmetry, we set:

$$\gamma_{\text{vis}}, \gamma_{\text{lang}} > \gamma_{\text{act}}$$

This ensures fragile backbone components are pulled back more strongly toward their aligned position after each task, while robust action components are allowed greater

freedom to adapt. We test three schedules for setting γ_m (in increasing order of sophistication):

1. **Fixed Schedule (Primary/Lowest-Risk):** Hand-tuned ratio via grid search on a held-out task sequence.
 2. **Drift-Informed Schedule:** γ_m set proportional to the empirical correlation between drift and behavioral forgetting for component m .
 3. **Online/Adaptive Schedule (Stretch Goal):** γ_m recomputed periodically from live drift measurements (EWC-like in spirit, but computed in latent space).
-

2. Alternate Way of Measuring Forgetting

Measuring forgetting behaviorally requires executing costly rollouts (simulated or physical) for every checkpoint/task combination under test. This does not scale. Our aligned latent space offers a purely representational alternative.

2.1 Latent Drift as a Forgetting Proxy

For component m of task k , after training through task $k + t$, we define *drift* as the Euclidean distance the task’s latent coordinate has moved from its originally aligned position. This requires no rollout and is computed directly from stored latent coordinates.

2.2 Validation Protocol

To establish whether drift is a valid proxy, we correlate it against ground-truth behavioral forgetting obtained via the (expensive) component-swap methodology. We swap the backbone or action head between checkpoints and measure the resulting success-rate drop, $\text{swap_drop}(m, k, t)$, on a constrained grid of task/checkpoint pairs.

We report the correlation coefficient between drift and swap_drop separately per component, hypothesizing that the relationship differs between vision-language and action components. Additionally, we run a **causal stress test**: deliberately varying γ_m across a range to check whether drift and behavioral swap_drop move together under intervention, providing stronger evidence than natural correlation alone.

2.3 Use as a Monitoring Signal

If validated, latent drift becomes a lightweight, rollout-free signal. It can be tracked continuously during training to flag when a task k ’s component m has moved too far and is at risk of behavioral forgetting—long before any physical or simulated re-evaluation is needed.

3. Memory and Benchmarking

3.1 Storage-Efficient Memory

Standard Experience Replay stores raw (o_t, a_t) transitions per task (hundreds to thousands of samples, including image observations), raising storage costs and privacy concerns. In contrast, our proposal’s memory footprint after training on task k is:

$$\text{Memory}(k) = \{e_{\text{vis}}^{(k)}, e_{\text{lang}}^{(k)}, e_{\text{act}}^{(k)}, Z^{(k)}\}$$

This consists of compressed embeddings and a latent token sequence—several orders of magnitude smaller than raw pixel/action trajectories, and not reconstructible into identifiable demonstration data. We report memory footprint in bytes/task alongside forgetting metrics, framing efficiency as a Pareto comparison (Normalized Backward Transfer vs. memory footprint).

(Note: Latent refinement still requires a small amount of real task k data to compute $\mathcal{L}_{\text{task}}$, making our method replay-light and storage-compressed, rather than fully replay-free.)

3.2 Model Zoo Construction

Before alignment training, we build a reference population of independently-trained LoRA adapters—one (or a few, multi-seed) per LIBERO task across all four suites (~40 tasks total). This follows WeightCLIP’s model-zoo protocol. This is a one-time cost and is necessary because contrastive alignment requires batch diversity to form meaningful negatives (a nontrivial constraint given LIBERO’s relatively small task count).

3.3 Sequential Evaluation Protocol

We adopt unmodified task orderings, backbones (Pi0, GR00T N1.5), and metrics established by prior continual-VLA work, across all four LIBERO suites (Spatial, Object, Goal, 10), at matched memory-footprint budgets.

- **Primary Metrics:** Success Rate (SR), Normalized Backward Transfer (NBT), Recovery Efficiency (steps-to-peak ratio), Memory Footprint (bytes/task), and Drift-to-Swap_Drop correlation.
- **Ablation Studies:**
 1. **Contrastive vs. Direct Regression:** Isolates whether alignment itself improves performance.
 2. **Component-wise vs. Uniform Regularization:** Isolates whether differentiation helps.
 3. **Fixed vs. Drift-informed γ_m :** Evaluates scheduling sophistication.
 4. **Direction-Inverted Ablation ($\gamma_{\text{act}} > \gamma_{\text{vis}}$):** Confirms that the *direction* of differentiation is what prevents forgetting, not just its presence.